BASE DE DATOS

Diseño y tecnologías de persistencia para 3Market

1. Objetivo

Definir el diseño y las tecnologías de la base de datos de 3Market: qué información se almacenará, cómo se organizará en tablas, qué relaciones existirán entre ellas y qué reglas garantizarán la integridad de los datos. La base de datos respalda los 14 requerimientos funcionales del sistema (clientes, catálogo, carrito, pedidos, pago simulado, inventario y reportes) dentro de la arquitectura monolítica MVC con Spring Boot.

2. Tecnologías seleccionadas

- **MySQL:** sistema gestor de base de datos relacional donde se almacenará toda la información de la plataforma.
- **Driver MySQL (JDBC):** conector oficial que permite a la aplicación Spring Boot comunicarse con MySQL.
- **Spring Boot Starter Data JPA / Hibernate:** mapeo objeto-relacional (ORM). Cada tabla se representa con una clase @Entity en el paquete model y se consulta mediante interfaces que extienden JpaRepository en el paquete repository.
- **Spring Boot Starter Validation:** validación de datos de entrada (@NotNull, @Size, @Email) antes de que lleguen a la base de datos.
- **Spring Security:** las contraseñas se guardarán cifradas (hash), nunca en texto plano, cumpliendo el RNF-03.
- **JasperReports:** los reportes de ventas se generan a partir de consultas sobre las tablas de pedidos y detalle de pedidos.

3. Qué se va a hacer con la base de datos

El trabajo sobre la base de datos se desarrollará en las siguientes etapas:

Etapa	Actividad	Resultado
1. Modelado	Identificar entidades, atributos y relaciones a partir de los requerimientos funcionales.	Modelo entidad-relación
2. Creación	Crear la base de datos en MySQL y sus tablas con claves primarias, foráneas y restricciones.	Script SQL de creación
3. Mapeo	Crear las clases @Entity y los repositorios JPA que representan cada tabla.	Paquetes model y repository

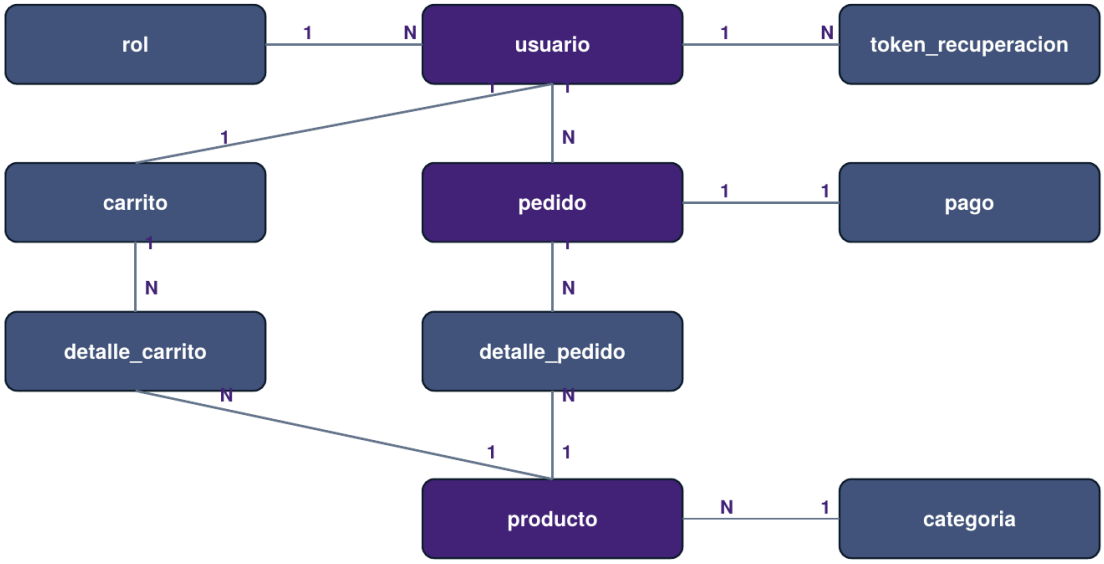
Etapa	Actividad	Resultado
4. Datos iniciales	Insertar los roles (Administrador, Vendedor, Cliente), un usuario administrador y categorías base.	Script SQL de datos iniciales
5. Lógica transaccional	Implementar en los servicios las operaciones críticas (compra, pago, descuento de inventario) con @Transactional.	Paquete service
6. Consultas y reportes	Preparar consultas de historial por cliente y de ventas por periodo, producto y categoría.	Reportes de ventas
7. Pruebas	Verificar integridad, restricciones, flujo completo de compra y consistencia del inventario.	Base validada

4. Modelo de datos propuesto

Se proponen las siguientes tablas principales. Los nombres de campos son una base de trabajo y podrán ajustarse según los acuerdos técnicos del equipo.

Tabla	Descripción	Campos principales
rol	Perfiles de acceso del sistema (Administrador, Vendedor, Cliente).	id (PK), nombre
usuario	Clientes, vendedores y administradores. Almacena las credenciales cifradas.	id (PK), nombre, apellido, correo (único), contraseña (cifrada), teléfono, dirección, activo, fecha_registro, rol_id (FK)
token_recuperacion	Códigos temporales para recuperar la contraseña (RF-03).	id (PK), token, fecha_expiracion, usado, usuario_id (FK)
categoria	Clasificación de los productos (RF-04).	id (PK), nombre (único), descripción, activa
producto	Catálogo de productos (RF-05, RF-06).	id (PK), nombre, descripción, precio, stock, imagen (ruta), activo, categoria_id (FK)
carrito	Carrito de compras activo de cada cliente (RF-08).	id (PK), fecha_creacion, usuario_id (FK)
detalle_carrito	Productos y cantidades dentro del carrito.	id (PK), cantidad, carrito_id (FK), producto_id (FK)
pedido	Orden de compra generada por el cliente (RF-09, RF-12, RF-13).	id (PK), fecha, estado, total, dirección_envío, usuario_id (FK)
detalle_pedido	Productos del pedido con el precio vigente al momento de la compra.	id (PK), cantidad, precio_unitario, subtotal, pedido_id (FK), producto_id (FK)
pago	Registro del pago simulado de la orden (RF-10).	id (PK), método, monto, estado, fecha_pago, pedido_id (FK)

4.1 Diagrama de relaciones



Notación: 1 = uno, N = muchos. Ejemplo: un usuario puede tener muchos pedidos; un pedido pertenece a un solo usuario.

5. Reglas de negocio e integridad

- **Claves primarias y foráneas:** toda tabla tendrá una clave primaria autoincremental y relaciones con integridad referencial; no podrá existir un detalle sin su pedido o carrito.
- **Unicidad:** el correo del usuario y el nombre de la categoría no podrán repetirse.
- **Inventario automático (RF-11):** al confirmar el pago de un pedido, el stock de cada producto se descuenta en la misma transacción; si algún producto no tiene stock suficiente, la operación se revierte por completo.
- **Estados del pedido (RF-12):** pendiente, pagado, enviado, entregado y cancelado. Si un pedido se cancela, el stock descontado se devuelve al producto.
- **Precio histórico:** el detalle del pedido guarda el precio del momento de la compra, de modo que un cambio posterior de precio no altera pedidos anteriores.
- **Borrado lógico:** productos, categorías y usuarios se desactivan en lugar de eliminarse físicamente cuando tienen historial, para conservar los reportes y el historial de compras.
- **Roles y permisos (RNF-03):** cada usuario tiene un único rol que determina a qué módulos puede acceder; las contraseñas se almacenan cifradas.

6. Relación con los requerimientos

Requerimiento	Tablas involucradas
RF-01, RF-02, RF-03 (registro, sesión, recuperación)	usuario, rol, token_recuperacion
RF-04, RF-05, RF-06, RF-07 (categorías, productos, catálogo, filtros)	categoria, producto
RF-08 (carrito)	carrito, detalle_carrito, producto

Requerimiento	Tablas involucradas
RF-09, RF-10 (orden y pago simulado)	pedido, detalle_pedido, pago
RF-11 (inventario automático)	producto, detalle_pedido
RF-12, RF-13 (estados e historial)	pedido, detalle_pedido, usuario
RF-14 (reportes de ventas)	pedido, detalle_pedido, producto, categoria

7. Configuración y organización en el proyecto

- La conexión a MySQL (URL, usuario, contraseña y dialecto) se configura en **application.properties**, sin dejar credenciales reales en el repositorio.
- Las entidades se ubican en **model/** y los repositorios en **repository/**, dentro del paquete com.unipacifico._3market.
- Los DTO y mappers evitan exponer las entidades directamente a las vistas Thymeleaf.
- Las imágenes de productos no se guardan en la base de datos: solo se almacena su ruta, y los archivos residen fuera del código fuente (por ejemplo, ./imagen/products).
- Los scripts SQL (creación y datos iniciales) se versionarán junto al proyecto para que todo el equipo pueda recrear la base de datos.

8. Alcance

Esta definición corresponde exclusivamente a la base de datos de 3Market: modelo de datos, tablas, relaciones, reglas de integridad, configuración de la conexión y mapeo con JPA. No incluye la implementación de las vistas del frontend ni la lógica completa de los controladores.

9. Consideraciones

- Mantener los nombres de tablas y campos coherentes y en un mismo estilo (por ejemplo, minúsculas con guion bajo).
- Usar transacciones (@Transactional) en toda operación que modifique varias tablas a la vez.
- Definir índices en los campos de búsqueda frecuente (correo, nombre de producto, categoría, estado y fecha del pedido).
- Validar los datos en la aplicación y reforzar con restricciones en la base de datos (NOT NULL, UNIQUE, CHECK de stock y cantidades no negativas).
- Realizar copias de seguridad periódicas de la base de datos, en apoyo a la disponibilidad 24/7 (RNF-02).
- Ajustar el modelo a los acuerdos técnicos del equipo antes de crear las tablas definitivas.

10. Estado

Diseño de base de datos propuesto para el desarrollo del proyecto. La implementación deberá ajustarse a los acuerdos técnicos del equipo y a la arquitectura y dependencias definidas en el documento del backend.